The ML System Design Rubric

Scaler School of Technology | ML System Design and LLMOps | Week 6 — Framework Reference

This rubric is a **structured 10-step framework** for answering any ML system design question. Follow the steps in order. Each step builds on the previous one. Interviewers evaluate your answer on **structure, completeness, and ability to articulate trade-offs** — not on proposing the most complex model.

1 Clarify the Problem

Ask questions before designing. Define scope, constraints, and success criteria. **Key questions**: What must the system achieve? What metrics define success? Are there time, data, or resource constraints?

2 Context

Provide domain and business context. Why does this system exist? What is the business objective? Who are the users? What happens if the system fails?

3 Problem Statement

Formally define the ML problem. Frame it precisely: what is the input, what is the output, and what does the model need to predict or rank?

4 Define Requirements

Split into two categories. **Functional**: what the system must do (personalisation, real-time inference, feedback collection). **Non-functional**: how it must behave (latency SLA, scale, availability, privacy, maintainability).

5 Estimations

Back-of-envelope sizing. Estimate: number of users, request volume (QPS), data volume per day, storage requirements, compute needed. These numbers drive architecture decisions.

6 System Architecture

Design the end-to-end system. Decompose into: **(a) Data Collection** — sources, signals, ingestion patterns; **(b) Data Engineering** — ETL, storage, schema; **(c) Feature Engineering** — features, feature store; **(d) Preprocessing** — candidate generation, data prep; **(e) Model** — algorithm choice, training pipeline, registry; **(f) Post-processing** — re-ranking, filtering, business rules.

7 Evaluation: Offline and Online

Define how you measure success. **Offline**: precision@K, recall@K, NDCG, AUC-ROC on held-out data. **Online**: CTR, session length, business north-star metric. Define guardrails (what must *not* degrade).

8 A/B Testing

Design the experiment to validate the model in production. Define: treatment and control groups, sample size, duration, statistical significance threshold, and how you will detect bias (e.g. selection bias, novelty effect).

9 Deployment and Monitoring

Describe the production lifecycle. **Pipelines**: scheduling, orchestration (Airflow, Kubeflow). **Serving API**: REST/gRPC, inference server (Triton, Vertex AI). **Model controller**: version management, blue-green or canary rollout, rollback. **Dashboards**: latency, drift, business metrics.

10 Risk Management and Trade-offs

Identify what can go wrong and what you gave up. Address: data bias and fairness, model interpretability vs. accuracy, staleness vs. cost, cold-start, failure modes, and ethical implications.

How to use this rubric in an interview: Work through Steps 1–3 aloud with the interviewer — they may constrain the problem further. For Steps 4–6 spend the most time: this is where architectural thinking is assessed. Steps 7–10 differentiate senior candidates. You do not need to cover every sub-bullet — but you must be able to justify every decision you make.

Case Study: LLM-powered Semantic Search at Scale

Scaler School of Technology | ML System Design and LLMOps | Week 6 — Reference Solution

Step 1 | Clarify the Problem

Given problem: Design a semantic search system for a large content platform (e.g., Stack Overflow, GitHub, internal enterprise knowledge base) with **100 million documents**.

Constraints given: Sub-500ms p99 latency • 5,000 QPS • LLM-augmented results • hybrid retrieval • new documents indexed within 1 hour.

Clarifying questions an interviewer would expect:

- What counts as a "result"? Snippet or full document? (*Assume: title + passage snippet + optional LLM summary*)
- Is the LLM summary always required, or only when helpful? (*Conditional on query intent — only for "answer" queries, not "find" queries*)
- What content domain and languages? (*English-only, developer/technical content*)
- What availability SLA? (*99.9%*)
- What is the LLM cost budget? (*$0.01 per query maximum*)

Step 2 & 3 | Context and Problem Statement

Context: Enterprise and developer search is a precision-first domain. A missed relevant document loses productivity time. A hallucinated LLM summary erodes trust, potentially more than no answer at all. The search bar is often the primary productivity interface — downtime or quality regression has immediate, measurable business impact.

Problem Statement: Given a natural language query, retrieve the top-K most relevant documents from a 100-million-document corpus and optionally augment with an LLM-generated answer. **Input:** query string. **Output:** ranked list of documents with metadata, plus an optional LLM-generated answer summary with citations. **Constraint:** p99 end-to-end latency $<$ 500ms, LLM cost $<$ $0.01 per query.

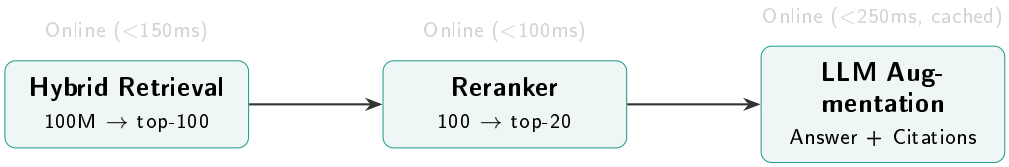
Step 4 | Define Requirements

Requirement	Implication for design
Functional	
Hybrid retrieval (dense + sparse)	Parallel ANN + BM25 with RRF merge; two index services
LLM-augmented answers	LLM in the serving path — must have timeout + fallback
New documents within 1h	Real-time indexing pipeline (Kafka + streaming ingest)
Query rewriting / intent detection	Pre-retrieval NLP stage before hitting the indices
Non-functional	
p99 $<$ 500ms end-to-end	LLM inference is the bottleneck — cache aggressively
5,000 QPS	Stateless serving pods; horizontal autoscaling
99.9% availability	Serving must not depend on training pipeline health
LLM cost $<$ $0.01/query	Only augment ~10% of queries; use smaller LLMs + caching

Step 5 | Estimations

- **Dense index:** 100M docs $\times$ 1536 dims $\times$ 4 bytes $\approx$ **600 GB**
- **BM25 index:** inverted index $\approx$ **50 GB**
- **Concurrent requests:** 5,000 QPS $\times$ 500ms $\approx$ 2,500 concurrent
- **LLM cost:** 1,000 tokens/query $\times$ $0.0002/1K = $0.0002/query; 10% of 5K QPS = 500 augmented/s = **$300/day**
- **Embedding new docs:** 100 docs/min $\times$ 1536 dims $\approx$ well within one GPU's capacity
- **Cache hit rate target:** 40% for popular queries — saves $120/day on LLM costs

The key architectural insight: You cannot run LLM inference over 100M documents per query. The solution is a **two-stage architecture**: fast hybrid retrieval narrows the corpus to top-K candidates (e.g., top 20), then the LLM augments only those results. LLM inference is the **latency and cost bottleneck** in the serving path — cache aggressively by query hash.



6a Data Collection

What it does: Ingests documents from crawlers, push APIs, and connectors (GitHub, Confluence, Jira, Stack Overflow).

Schema per document: `doc_id`, `title`, `body`, `author`, `created_at`, `updated_at`, `doc_type`, `source`, `tags`.

Key design: Kafka topic for real-time ingest of new/updated documents. Schema validation on ingest prevents corrupt embeddings. Separate *cold* (bulk historical) and *hot* (streaming new) pipelines.

6b & 6c Data Engineering & Feature Engineering

Chunking strategy: Split documents into 512-token chunks with 50% overlap. Small chunks ⇒ lost context; large chunks ⇒ noisy retrieval. Overlap preserves sentence continuity.

Deduplication: MinHash LSH on document bodies to remove near-duplicates before indexing.

Dense embeddings: `text-embedding-3-small` (OpenAI) or self-hosted `E5-large`. **Critical**: the embedding model used at index time and query time must be identical — version mismatch causes silent retrieval collapse.

Sparse index: Elasticsearch BM25 index over title + body tokens. Handles exact-match keyword queries that dense models miss.

Metadata filters: date range, document type, author, source system — applied as pre-filters before ANN search.

6d & 6e Preprocessing & Models

Query understanding (pre-retrieval): Intent classifier (answer vs. find vs. navigate). Query rewriter to expand abbreviations and normalise. Language detection.

Hybrid retrieval: Dense ANN (FAISS/Pinecone) and BM25 (Elasticsearch) run **in parallel**. Reciprocal Rank Fusion (RRF) merges both result lists into a single ranked list of top-100.

Reranking: Cross-encoder (e.g., `ms-marco-MiniLM-L12`) rescores top-100 → top-20. Cross-encoders read query and document together — much higher precision than bi-encoders but too slow for 100M docs.

LLM augmentation: GPT-4o-mini (or equivalent) generates a 3–5 sentence answer citing the top-3 documents. **Only triggered when** intent classifier returns "answer". Result cached by MD5 of the normalised query.

6f Post-processing & Serving

Latency budget (p99 target = 500ms):

1. Network + load balancer: 10ms
2. Query understanding (intent + rewrite): 20ms
3. Parallel ANN + BM25 retrieval: 150ms
4. Cross-encoder reranking (top-100): 100ms
5. LLM answer generation (cached): 220ms / 0ms on cache hit

Safety filter: LLM output passes through a content safety classifier before serving. **Fallback:** if LLM times out (>300ms) or returns unsafe content, serve ranked results only (no summary). Users should not perceive a failure.

Step 7 | Evaluation: Offline and Online

Type	Metric	Why it matters
Offline	NDCG@10	Penalises relevant documents ranked lower; primary retrieval quality signal
	MRR (Mean Reciprocal Rank)	How quickly does the first relevant result appear?
	Recall@100	Is the answer-bearing document anywhere in the top 100? Prerequisite for reranking.
Online	**CTR on results (north star)**	Did the user click a result? 7-day rolling average
	LLM answer satisfaction	Thumbs up/down on generated summaries
	Zero-result rate	Fraction of queries with no clicks — signals recall failures
	Query abandonment rate	User left without interacting — signals all results are irrelevant

Guardrail metrics: LLM p99 latency must stay < 300ms. Hallucination rate (spot-checked by human raters or automated NLI classifier) must remain $< 2\%$. LLM cost per query must stay below \$0.01. A system that gives fast, wrong answers destroys user trust faster than slow, correct ones.

Step 8 | A/B Testing

Experiment design:

- **Unit of randomisation:** User session (search session ID) — query-level randomisation risks within-user contamination
- **Treatment options:** (a) LLM-augmented vs. no augmentation; (b) cross-encoder reranking vs. no reranking; (c) new embedding model vs. current
- **Traffic split:** 10% treatment initially; 50/50 after 48h with no guardrail violations
- **Duration:** 14 days minimum to capture weekly usage patterns and wash out novelty effect
- **Bias controls:** Watch for *novelty effect* — users explore more when the interface changes. Watch for *selection bias* on query difficulty: if treatment gets harder queries, CTR may appear lower even if quality improved. Match treatment and control query distributions.
- **Decision rule:** Promote if CTR improves $\geq 1\%$ **and** LLM cost per query stays $\leq$ \$0.01 **and** no guardrail degrades. Rollback if p99 > 600ms or hallucination rate $> 2\%$.

Step 9 | Deployment and Monitoring

Pipelines

- Airflow for nightly full reindex jobs
- Kafka consumer for real-time doc ingest ($<$ 1h SLA)
- MLflow tracks embedding model versions
- Automated evaluation gate: new embedding model must meet Recall@100 threshold on benchmark before promotion

Model rollout

- Embedding model upgrade requires **full reindex** of 100M docs — expensive, plan carefully with canary reindex on a 1% sample first
- Blue-green for reranker and LLM prompt changes (no reindex needed)
- LLM answer cache (Redis, TTL = 24h) — survives pod restarts

6 Key Metrics to Monitor:

1. **p99 query latency by stage** — retrieval, rerank, LLM separately
2. **Index freshness lag** — minutes since last document was indexed
3. **Cache hit rate** — target $> 40\%$ for LLM augmentation
4. **LLM API error rate** — triggers fallback to non-augmented results
5. **Embedding model version skew** — index vs. serving version must match
6. **Zero-result rate** — alert if $> 2\%$ of queries return no clicks

Decision	Choice	Alternative	Why this choice
Retrieval method	Hybrid (dense + sparse)	Dense only	Dense is better for semantic; sparse is better for exact keywords (e.g., error codes, function names). Hybrid outperforms either alone.
LLM augmentation	Conditional (intent = answer)	Always augment	Always = higher cost + latency on every query. Conditional = cheaper but requires intent classifier. Worth the complexity.
Embedding model	Self-hosted E5-large	OpenAI API	API is simpler but creates vendor lock-in and latency variability. Self-hosted gives control; costs more ops.
Chunking strategy	512 tokens, 50% overlap	Full document or small 128-token	Too small = lost context; too large = noisy retrieval. 512 with overlap is the empirically robust default.
Reranking	Cross-encoder on top-100	No reranking	Cross-encoder adds 100ms but lifts NDCG@10 by $\sim$15%. Fits within 500ms budget.
Embedding upgrade	Canary reindex (1% first)	Big-bang reindex	Full reindex of 100M docs takes hours. Canary lets you validate quality before committing.

Failure Modes and Recovery

What breaks	How to detect	Recovery
Embedding model version mismatch	Retrieval quality collapses silently; NDCG drops, no latency change	Compare index embedding model version tag vs. serving model version in every request; halt serving if mismatch; trigger reindex
LLM timeout	p99 latency spikes above 500ms	Hard timeout at 300ms; fallback to non-augmented ranked results; page on-call; degrade gracefully
Index staleness (hot content)	Freshness lag metric rises; users search for recent documents and get no results	Monitor lag per source; alert if $>$ 90min; check Kafka consumer lag
Cache poisoning (bad LLM answer cached)	Thumbs-down rate spikes on a particular query	TTL-based invalidation (24h max); feedback-driven purge via admin API; add NLI fact-check layer
BM25 index unavailable	Elasticsearch health check fails; hybrid retrieval degrades to dense-only	Circuit breaker: fall back to dense-only ANN; alert; do not serve empty results